Class 2: Software Systems — Frontend to Deployment

From Zero to AI Agents — Seven-Class Edition

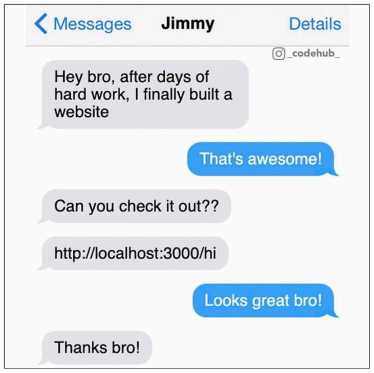
Scan to open the Class 2 document

Shared Google Doc

From a Program to a Product

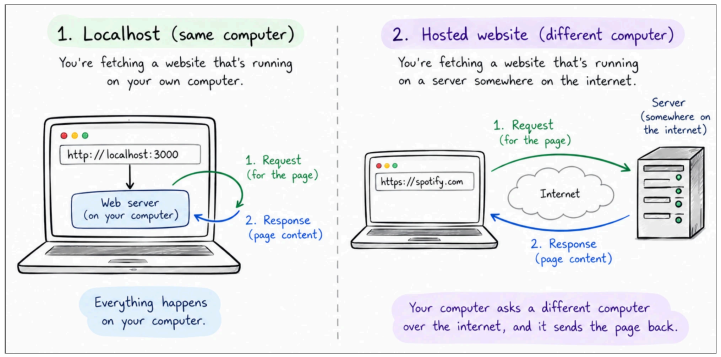
Localhost Is for Testing, But You Can't Share It

Last class you built a pricing function, tests around it, and a small report site. All of it ran on `localhost`, which just means your own laptop.
Hit Ctrl+C and it's gone. Send the link to a coworker and they get nothing, because that address points at *their* machine, not yours.



Today we fix that. Same logic, but on a **real URL anyone can open**, with real data behind it, access control, and a deploy that keeps running after you close the lid.

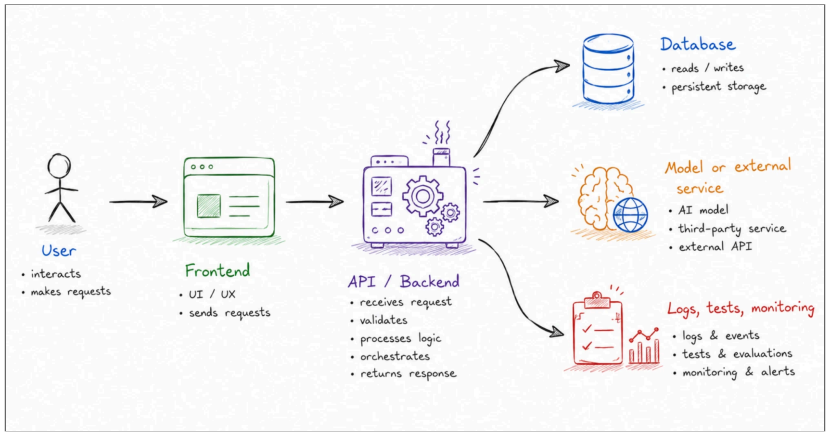
What Happens When You Type a URL



Both sides are the same two steps: your browser asks for a page, something sends one back. The only difference is who answers. On `localhost` that's your own laptop talking to itself. On a hosted site it's a machine anyone can reach.

> **Tip**
>
> **This is today's map. Frontend** is what your browser does with the page it gets back. **Backend** is the machine on the right answering the request. **Databases** are what that machine reads and writes. **Secrets & auth** are how it knows who is asking. **Cloud & deployment** are how it comes to exist in the first place.

The Product Stack



Every app you use — Instagram, Uber, your bank — is this diagram with more boxes. By the end of today, you'll have touched every layer *and shipped one to the internet*.

Today's Agenda

1. **Frontend** — HTML, CSS, JavaScript (live, editable, in this deck)
2. **Backend** — servers, APIs, and the waiter metaphor
3. **Databases** — durable shared memory, plus live SQL
4. **Secrets & identity** — API keys, env vars, authentication
5. **The cloud** — someone else's computers
6. **Testing & deployment** — from your laptop to a live URL
7. **Tech stacks** — how the pieces get chosen
8. **Assignment 1** — build and deploy a networking tracker

> **Note**
>
> **A note on pacing.** Slides badged **SELF-STUDY** are real material, not filler — they're the operational depth you'll want when you actually build this. We may skip them live to protect time for the hands-on parts. They stay in the published deck; read them when the question comes up at work.

Frontend

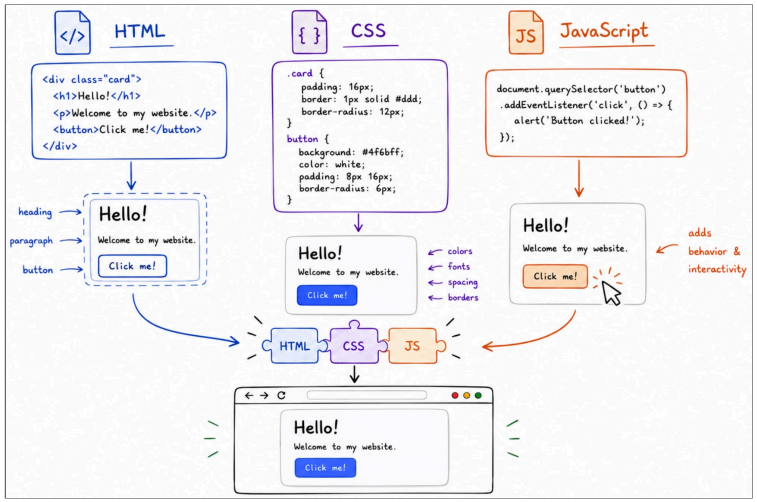
Frontend — What the User Sees

When you open a website, everything you see and interact with — the buttons, text, images, animations — that's the **frontend**. It runs in your browser (Chrome, Safari, Firefox).



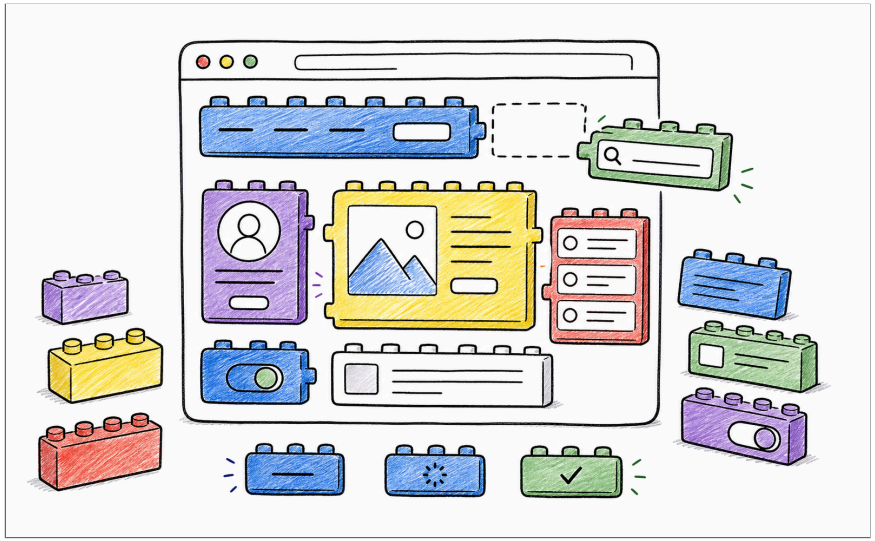
The frontend's job: turn a human intention into an understandable interface and a **valid request to the backend**.

How They Stack Together



Modern Frontend Frameworks

Frameworks give developers reusable building blocks and a structure for making interactive apps.
They keep the screen in sync as data changes, so you do not have to wire everything from scratch.





React



Next.js

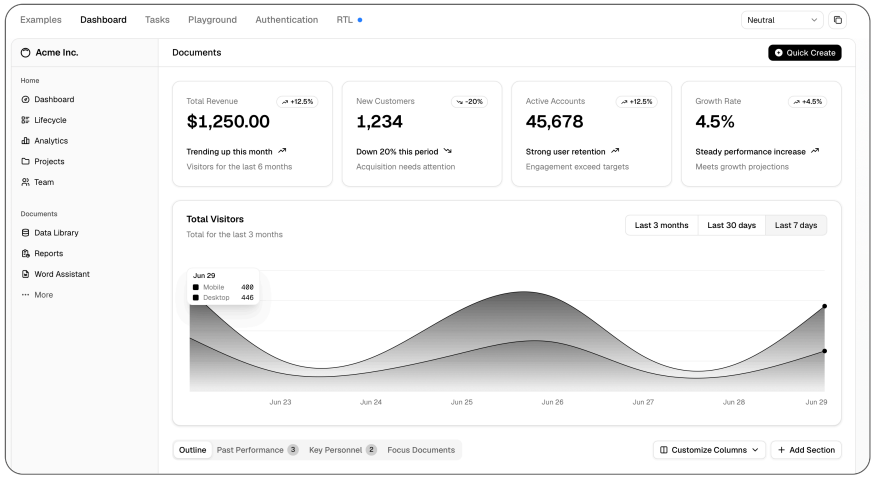
Two common choices for building component-based interfaces.

Component Systems Help You Start Ahead

Online component systems provide ready-made interface pieces you can install, copy, and adapt.

- Start with working buttons, forms, dialogs, tables, and page layouts.
- Keep the interface consistent across the product.
- Spend more time on what makes your app unique.

> **Prime example: shadcn/ui**
> Its CLI copies the component source into your project. You own the code and can customize every detail.
>
> `npx shadcn add dashboard-01`



A complete dashboard assembled from reusable shadcn/ui components.

Start with a Web App, Then Build a Mobile App

Web and mobile app frameworks ship differently, but your web app should still be responsive and mobile-friendly.

Web frameworks

Built for browsers and shared through a link.



React



Next.js



Vue



Angular



Svelte

Mobile frameworks

Built for installed apps and native platform behavior.



React Native



Expo

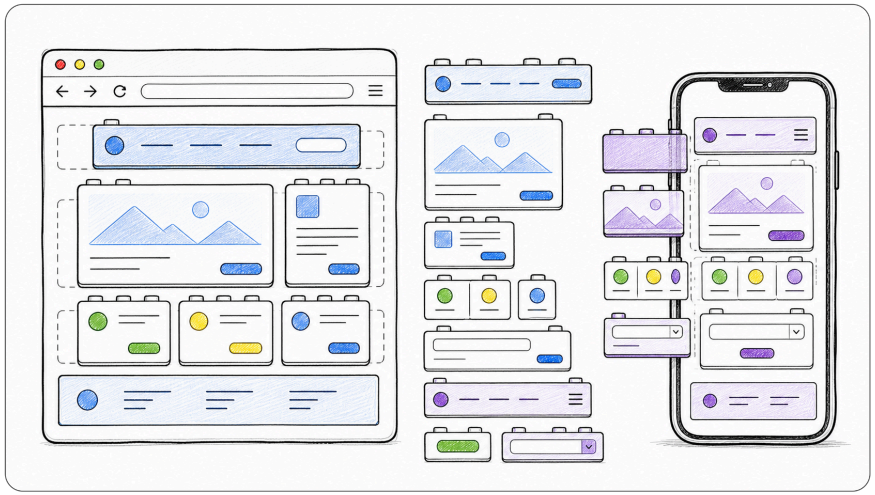


Flutter



SwiftUI

Jetpack Compose



My recommendation: build the web app first. It is the fastest way to share the product, test whether people want it, and improve it without an app-store release. Build a mobile app when native capabilities, deeper offline use, or app-store distribution become central to the product.

Backend

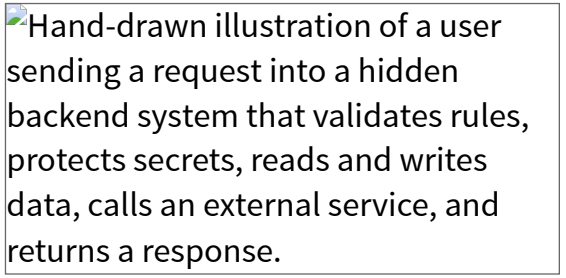
Backend: The Brain Behind the App

The frontend is what you see. The **backend** is the brain behind it. It decides what happens, enforces the rules, and keeps your secrets safe.

When you open Instagram, the frontend shows photos. But the backend decided **which** photos to show you, **checked you're allowed** to see them, and **billed the advertiser** who paid to be in your feed. All invisible to you.

The backend is not the database. It validates requests, applies business

rules, reads and writes data, calls external services, and returns an intentional response.


Hand-drawn illustration of a user sending a request into a hidden backend system that validates rules, protects secrets, reads and writes data, calls an external service, and returns a response.

Why Do You Need a Backend?

- **Centralized logic** — The rules of your business live in one place. Change a pricing formula once, it updates everywhere (mobile, web, API).
- **Controlled access** — Your secret sauce stays on the server. Users get results, not your algorithms or data.
- **One version of truth** — All devices talk to the same backend. No sync conflicts, no stale data.
- **Security** — API keys, passwords, and business logic never leave the server. The frontend is public — anyone can inspect it.

The Spotify Example

Imagine Spotify with **no backend**:

- Your phone has a playlist. Your laptop has a different one.
- You buy Premium on your phone — your laptop doesn't know.
- Your friend can't see your playlist at all.
- Anyone could reverse-engineer the app and get free music.

The backend solves all of this. It's the **single source of truth** that every device talks to.

Multiple Frontends, One Backend

> **Important**
>
> Mobile, web, TV, and watch are different frontends for the same system. The backend is the **one brain** they all share. Change the logic once, and every device gets the update.

APIs: The Waiter Metaphor

Tables are frontends. Waiters are APIs. The kitchen is the backend. The fridge is the database. Customers never enter the kitchen or open the fridge themselves.

An API is an Agreement Between Systems

```
POST /quotes
{ "customer_id": "cus_42", "revenue": 120000 }
```

The API **contract** says what callers can send and what they can rely on receiving.

Version contracts deliberately. Changing a field name or response shape can break every consumer — the mobile app, a partner's integration, a dashboard nobody remembers building. Evolve APIs with compatibility, migrations, and a communicated retirement path.

HTTP Methods

When your browser talks to a server, it uses **HTTP methods**:

Method	Purpose	Example
GET	Read data	"Show me user #42's profile"
POST	Create data	"Create a new account"
PUT	Update data	"Change my email address"
DELETE	Remove data	"Delete this photo"

> **Note**
>
> You'll also meet **PATCH** — change *part* of a resource (just the email, not the whole profile). The verbs communicate **intent**: a well-designed API reads like a sentence.

JSON — The Language of APIs

APIs send data as **JSON** (JavaScript Object Notation) — a simple text format that both humans and computers can read:

```
{
    "name": "Alice",
    "age": 25,
    "hobbies": ["coding", "hiking", "chess"]
}
```

Look familiar? It's the "list of dictionaries" shape from Class 1, as text. JSON is how state travels between machines.

Simulating a Server in Python

Python Code | Start Over | Run Code

```python
import json

users_db = [
    {"id": 1, "name": "Alice", "role": "engineer"},
    {"id": 2, "name": "Bob", "role": "designer"},
    {"id": 3, "name": "Charlie", "role": "data scientist"}
]

def api_server(method, path, body=None):
    """A simple simulated API server"""
    if method == "GET" and path == "/users":
        return 200, users_db

    if method == "GET" and path.startswith("/users/"):
        user_id = int(path.split("/")[-1])
        user = next((u for u in users_db if u["id"] == use
        if user:
            return 200, user
        return 404, {"error": "User not found"}

    if method == "POST" and path == "/users":
        new_user = json.loads(body)
        new_user["id"] = len(users_db) + 1
        users_db.append(new_user)
        return 201, new_user

    return 400, {"error": "Unknown route"}
```

```
_pyodide_editor_1 = ▸Object {code: null, options: Object, indicator: Ge}
```


Testing Our Server

Python Code | Start Over | Run Code

```python
print("=== GET /users ===")
status, data = api_server("GET", "/users")
print(f"Status: {status}")
print(json.dumps(data, indent=2))

print("\n=== POST /users ===")
status, data = api_server("POST", "/users",
    '{"name": "Diana", "role": "PM"}')
print(f"Status: {status}")
print(json.dumps(data, indent=2))
```

```
_pyodide_editor_2 = ▸Object {code: null, options: Object, indicator: Ge}
```

> **Tip**
>
> In production, you'd use a framework like **FastAPI** or **Flask** (Python), **Express** (Node.js), or **Gin** (Go) to build real API servers.

Never Trust Incoming API Requests

Never let an untrusted request directly become a database query, a shell command, a billing action, or a model tool call. Our toy server above trusts its input completely. Watch what a hostile caller could do:

```
# POST /users with body: '{"name": "Eve", "role": "admin", "id": 1}'
# → overwrites nothing here, but in a naive real system:
#   "role": "admin"  → privilege escalation
#   "id": 1          → identity takeover
```

> **Important**
>
> Rule of thumb: the backend treats **every** incoming request as potentially hostile — even from your own frontend, because anyone can send requests that never touched your frontend. Validate types, ranges, permissions, and *which fields the caller is even allowed to set*.

Databases Give Apps Memory

Every app needs somewhere durable to store its users, posts, messages, and orders.
Without a database, closing an app would erase its memory.

A Database Organizes Records

Think of a database as an organized warehouse: every kind of record has a defined place and a reliable way to find it.

Database = warehouse · **Table** = cabinet · **Row** = one record card · **Column** = one consistent field · **ID** = barcode

Tables Store Facts. Relationships Connect Them.

Tables define one kind of thing. Every row follows the same columns, while users, playlists, and songs belong in separate tables because they describe different entities.

Relationships reconnect those rows through IDs. Together, the tables and their links form the **data model** — the map to design before writing the code.

Spotify Uses All Three Relationship Types

A user has one profile, creates many playlists, and builds those playlists from songs.

One-to-one: user ↔ profile · **One-to-many:** user → playlists · **Many-to-many:** playlists ↔ songs through `playlist_songs`

SQL — Talking to Databases

SQL (Structured Query Language) is how you ask a database for data:

```sql
-- Get all users
SELECT * FROM users;

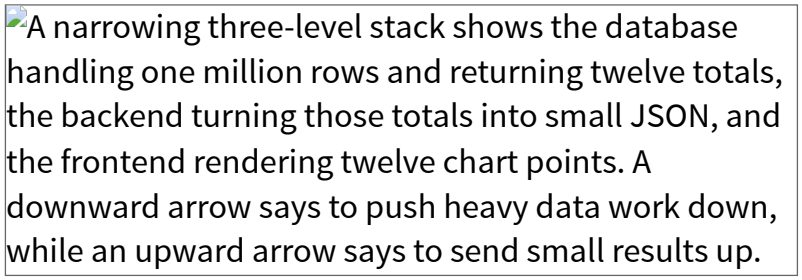
-- Get only engineers
SELECT name, email FROM users WHERE role = 'engineer';

-- Count users by role
SELECT role, COUNT(*) FROM users GROUP BY role;

-- Sort by name
SELECT * FROM users ORDER BY name;
```

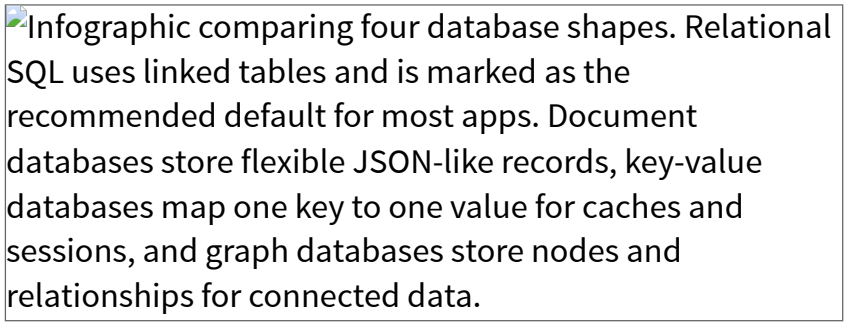
> **Note**
>
> **AI is exceptionally good at writing SQL now.** Give it your table schemas and describe the question in plain English; it can produce complex joins, filters, and aggregations for you in seconds. You still need to understand the result well enough to check it before running it on production data.
>
> SQL has been around since the 1970s and is used by virtually every company. Learning enough SQL to ask clearly and verify the answer is one of the **highest-ROI skills** in tech.

Push Big Data Work Down the Stack

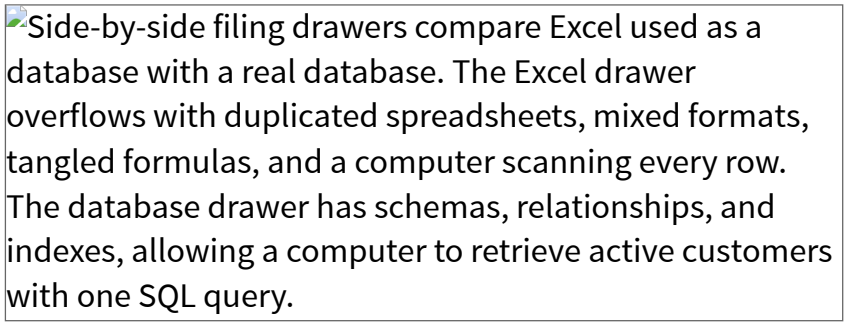


For large-data work: database > backend > frontend. Filter, join, sort, and aggregate next to the data; apply business rules in the backend; send the browser only the small result it needs to render.

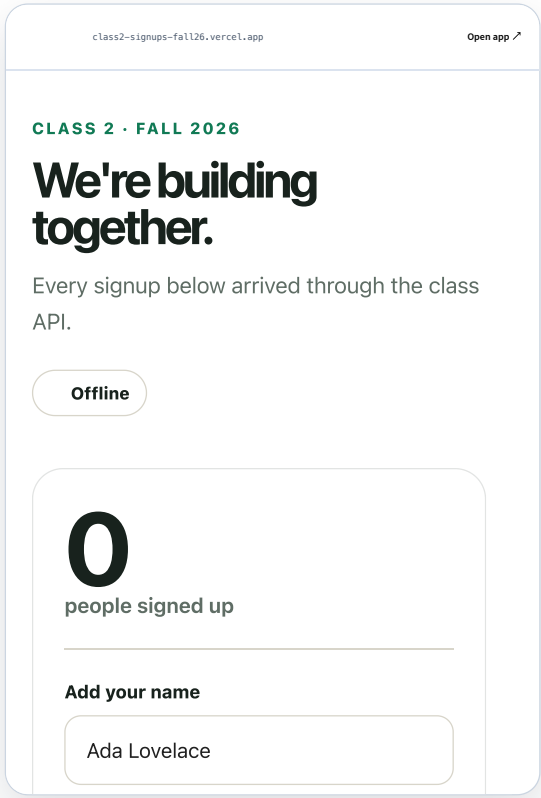
Types of Databases



Why Excel Is a Terrible Database



Live App: Class 2 Signups



What Is a Tech Stack?

A **tech stack** is the combination of technologies used to build and run an application.

Frontend

What users see and interact with:

- **Next.js:** frontend and backend in one project
- **React + Vite:** a clean, separate frontend
- **Vue:** a gentler learning curve

Backend

Business logic, APIs, security, and integrations:

- **Next.js API routes:** backend in the same project
- **FastAPI:** Python, AI, and data products
- **Express:** minimal JavaScript, huge ecosystem
- **Go:** concurrency and performance

Data

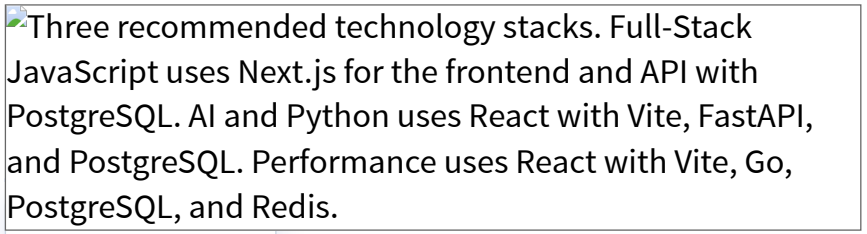
The application's durable memory and fast temporary storage:

- **PostgreSQL:** the default for most apps
- **MongoDB:** flexible document data
- **Redis:** fast caching and sessions beside a main database

> **Note**

> The signup app uses **Next.js + React**, **Next.js API routes**, **Supabase PostgreSQL**, and **Vercel**. Choose popular tools that fit your team's language and workload.

Three Practical Stack Recommendations



ck JavaScript for simplicity. Choose
oduct depends on AI or data science.
dis only when performance is a proven

Setup guide Scan to follow along Open instructions ↗

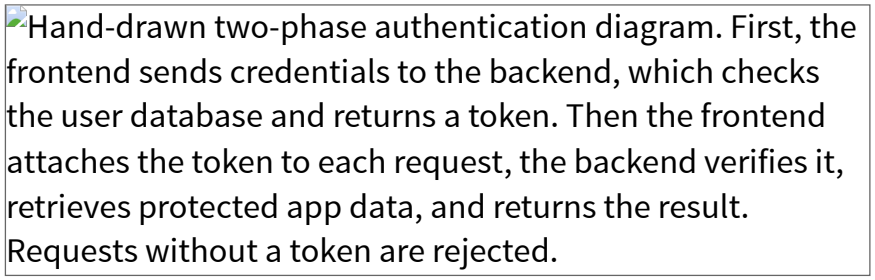
Authentication

Why Authentication Matters

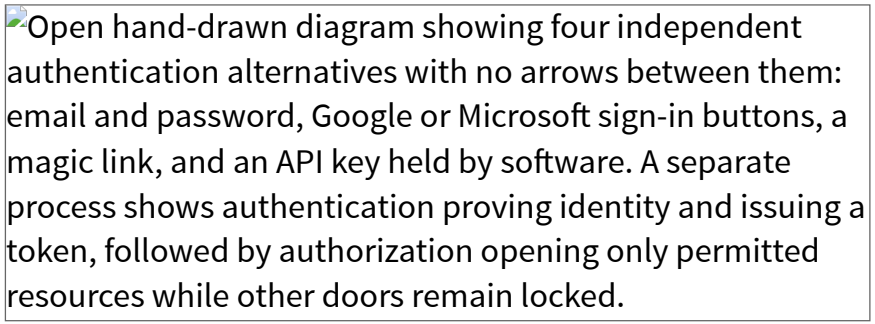
Without authentication, anyone can access anyone’s data. Your app has no idea who is making a request.
Authentication answers one question: **“Are you who you say you are?”**

> **Important**
>
> If you build a prototype with no auth, every user can see every other user’s data. This is the #1 mistake in vibecoded apps.

How It Works



Ways to Prove Identity and Control Access



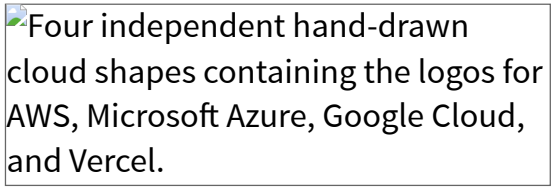
Cloud Platforms

What is “The Cloud”?

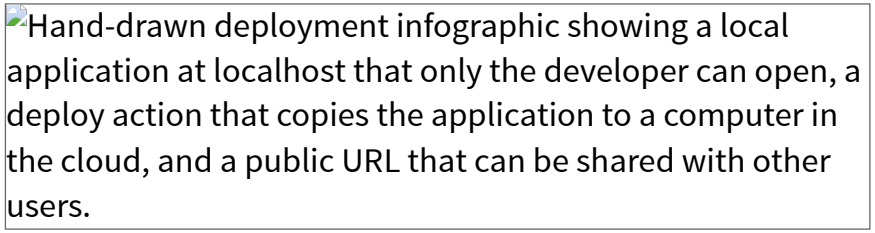
The cloud is **someone else’s computers in a data center**.
Instead of buying and maintaining servers, you rent computing power from a cloud provider.

- Runs 24/7, even when your laptop sleeps
- Available to users through the internet
- Can scale as traffic grows
- The provider manages the physical hardware

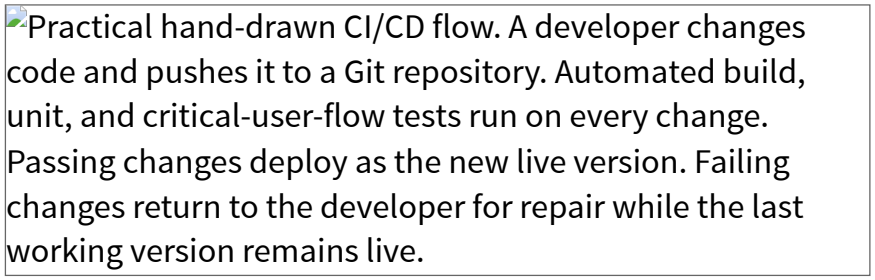
You deploy the code. They keep the computers running.



Deploying Makes Your App Available to Users

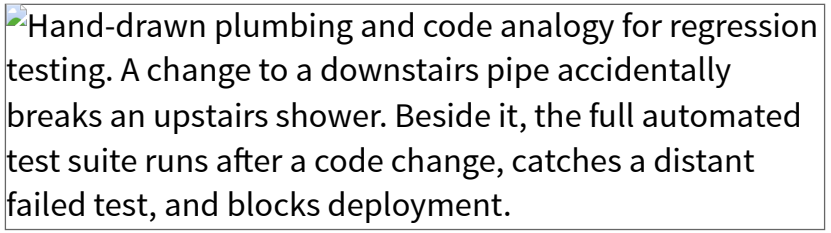


CI/CD Uses Tests as a Release Gate


Practical hand-drawn CI/CD flow. A developer changes code and pushes it to a Git repository. Automated build, unit, and critical-user-flow tests run on every change. Passing changes deploy as the new live version. Failing changes return to the developer for repair while the last working version remains live.

Regression Tests Catch Distant Breakage

A **regression** is when a change accidentally breaks something that already worked. Regression tests rerun the full suite after every change, catching failures far from the code you touched before users find them.


Hand-drawn plumbing and code analogy for regression testing. A change to a downstairs pipe accidentally breaks an upstairs shower. Beside it, the full automated test suite runs after a code change, catches a distant failed test, and blocks deployment.

Keep Secrets Out of Your Code

What goes wrong

- You hard-code an API key or password
- You push it to GitHub, where Git history remembers it
- Or you ship it in frontend code, which every browser can inspect
- An attacker can use your account, access data, or create charges

The safe pattern

- **Local:** store secrets in `.env.local`
- **Production:** use your cloud provider's environment settings
- The **backend** reads secrets at runtime
- The frontend calls your backend and **never receives the secret**

> **Important**
>
> 1. **Never commit `.env` files**
> 2. **Never put secret keys in frontend code**
> 3. **If a secret leaks, revoke and rotate it immediately**

The Assignment

> **Important**
>
> **Build a networking tracker:** a website to record the people you need to network with at Berkeley.
>
> - Sign in before using the app
> - Add, view, edit, and delete your own contacts
> - Keep every user's contacts private from every other user
> - Validate inputs and include at least one automated test
> - Deployed live on the internet
> - Submit one public GitHub repository URL with all grading evidence in its README
>
> **Stack:** Next.js + Supabase + Vercel

QR code for the Secure Networking Tracker assignment guide.

Full assignment guide
Open Google Doc ↗
README, setup, authentication, RLS, testing, deployment, and submission evidence.

Definition of Done: Your Rubric

Submit one public GitHub repository URL. Put the live URL and every evidence item below in its README.

Check	Evidence
The app is **live** on a public URL	README: live URL
A user can **sign in and sign out**	README: screenshot or recording
You can **add, view, edit, delete** a contact and it survives refresh	README: screenshot or recording
User A **cannot see or change User B's contacts**	README: two-account test evidence
One **invalid input fails safely** (empty name, bad priority)	README: error screenshot
At least one **automated test passes**	README: test output
Secret keys are in **server-only environment settings**, not frontend code	README: publishable versus secret-key explanation

Check	Evidence
You can explain the **schema and RLS ownership rule**	README: schema and `user_id` ownership rule

> **Tip**
>
> The security, test, and explanation rows are the difference between "an AI built me a demo" and "I shipped software I understand."

Congratulations — You Are Now AI Engineers

You walked in not knowing what a server was.
You just:

- Edited live HTML, CSS, and SQL in your browser
- Simulated an API server and queried a database
- Used an AI coding agent
- Built a full-stack web app
- Deployed it live to the internet

> **Important**
>
> **That’s not a tutorial. That’s software engineering.**
>
> Welcome to the club. See you in Class 3.

Next Class

Software follows rules you write. **Machine learning learns a rule from examples** — powerful, useful, and much easier to misuse.

Next: Class 3 — Machine Learning Foundations →

Optional Self-Study

These slides are optional reference material for deeper study.

Status Codes Are Part of the Contract SELF-STUDY

Every response comes with a three-digit status code — the server's one-line summary of what happened:

Code	Meaning	Whose problem?
200	Succeeded	Nobody's — celebrate
201	Created something new	Nobody's
400	The caller sent a bad request	The client's
401	Not signed in	The user's (or the auth flow's)
403	Signed in but not allowed	Permissions
404	No such thing here	The URL's
500	Our system failed	**The server's — page someone**

> **Tip**
>
> The families matter more than the numbers: **2xx** = success, **4xx** = the caller messed up, **5xx** = we messed up. When your team says "we're seeing 500s," that's the ops equivalent of a fire alarm.

Anatomy of an HTTP Request SELF-STUDY

The whole thing is just structured text. Here's what your browser actually sends:

```
POST /api/contacts HTTP/1.1              ← method, path, protocol
Host: networking-tracker.vercel.app      ← which site
Authorization: Bearer eyJhbGc...         ← who you are (the token!)
Content-Type: application/json           ← what format the body is

{"name": "Maya Chen", "company": "Stripe", "priority": "high"}
```

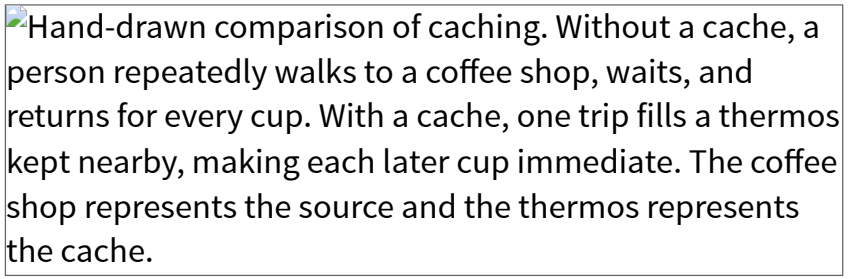
And what the server sends back:

```
HTTP/1.1 201 Created                     ← status code
Content-Type: application/json

{"id": 17, "name": "Maya Chen", "company": "Stripe", "priority": "high"}
```

> **Tip**
>
> Open your browser's DevTools → Network tab and click around any website. Every request looks exactly like this. Once you can read it, no web app is a black box to you again.

What Is a Cache?


Hand-drawn comparison of caching. Without a cache, a person repeatedly walks to a coffee shop, waits, and returns for every cup. With a cache, one trip fills a thermos kept nearby, making each later cup immediate. The coffee shop represents the source and the thermos represents the cache.

Caches Trade Freshness for Speed SELF-STUDY

Caching is not free performance. It is a decision about **how stale an answer is allowed to be**.

- Stock price on a trading app: stale after 1 second
- Product catalog: stale after an hour, probably fine
- Company logo: cache it until the rebrand

> **Note**
>
> There's an old joke: *"There are only two hard things in computer science: cache invalidation and naming things."* Both are jokes because both are true.

Backups Answer a Different Question Than Availability SELF-STUDY

Replication keeps a live system running when a server dies. **Backups** let you recover from deletion, corruption, or a bad migration.
Replication faithfully copies your mistake to every replica in milliseconds. Only a backup holds yesterday's truth.

> **Important**
>
> Good teams **rehearse restoration**. An untested backup is a hope, not a plan. The question to ask your team isn't "do we have backups?" — it's "when did we last restore one, and how long did it take?"

Latency: Why Apps Feel Slow SELF-STUDY

Distance and layers have a price. Rough numbers worth having in your gut:

Operation	Time	Human terms
Read from memory (cache hit)	~0.0001 ms	instant
Query a database (indexed)	~1–10 ms	instant
Query a database (missing index, big table)	100 ms – 10 s	“is it frozen?”
API call, same data center	~1–5 ms	instant
API call across an ocean	~150 ms	noticeable
Call an LLM API	1–30 s	coffee-adjacent

> **Tip**
>
> This table explains half of system design: caches exist because memory beats the database; CDNs exist because the speed of light is a real constraint; and “the AI feature feels slow” is physics plus model size, not laziness. When someone proposes an architecture, ask **how many of these hops sit between the click and the answer**.

Build vs Buy: The Managed-Services Ladder SELF-STUDY

Every capability can be bought at several altitudes:

Level	Example	You manage	You pay
Bare metal	Your own servers	Everything	Hardware + people
IaaS	AWS EC2	OS, database, scaling	By the hour
PaaS	Vercel, Railway	Just your code	By usage
SaaS	Supabase, Auth0, Stripe	Just the configuration	By subscription

The pattern of the last 20 years: **keep moving up the ladder** until a specific need (cost at scale, compliance, control) forces you down.

> **Important**
>
> For prototypes and internal tools, the answer is almost always the top of the ladder. Engineering pride is not a business reason to run your own Postgres. The exceptions are real — data residency, unit economics at massive scale — but they are *exceptions*, and they come with headcount attached.

The Test Pyramid, in Plain English SELF-STUDY

- **Many fast checks of small decisions** — unit tests (“does `calculate_discount` handle the overdue case?”)
- **Fewer integration checks of system boundaries** — (“does the API actually write to the database?”)
- **A few end-to-end checks of the user’s real path** — (“can a user sign up, add a contact, and see it?”)

> **Tip**
>
> The shape matters because of cost: unit tests run in milliseconds and pinpoint the failure; end-to-end tests take minutes and tell you only “something broke somewhere.” Buy lots of the cheap ones and a few of the expensive ones.

Environments Are Intentionally Different Places

SELF-STUDY

Deployment is moving a **tested version** into an **environment**:

Environment	Who uses it	Data	Credentials
Development	You, on your laptop	Fake / local	Test keys
Preview	Your team, per-PR	Fake or copy	Test keys
Production	Real users	Real	Live keys

> **Important**
>
> Development, preview, and production should have **intentionally different credentials**. “The prototype was accidentally pointed at prod” is a sentence you never want to say out loud.

Rollback: The Undo Button You Design in Advance

SELF-STUDY

Deploys will go wrong. The question is how long users suffer while you fix it.

- **Roll forward** — push a fix on top. Fine for typos; slow for disasters.
- **Roll back** — redeploy the previous known-good version. On Vercel, it’s one click: “Promote previous deployment.”
- **Feature flags** — ship the code dark, flip a switch to enable it, flip back to disable. Decouples *deploying* from *releasing*.

> **Tip**
>
> The mature question is not “will deploys break?” but “what is our **time to recover**?” Elite teams measure it in minutes — not because they break less, but because undo is cheap.

Alerts Should Point to Action SELF-STUDY

An alert that nobody can interpret at 2 a.m. is just anxiety with a webhook.
Good alert: *“Error rate on /checkout above 5% for 10 minutes — runbook: link.”* Bad alert: *“Something is not OK.”*

Production Readiness is a Checklist, Not a Feeling

SELF-STUDY

Before launch, name:

- the **owner** — who gets paged
- the **access policy** — who can see and change what
- the **rollback path** — how you undo a bad deploy
- the **health check** — how a machine knows it’s alive
- the **alert** — how a human finds out it isn’t
- the **most likely failure mode** — and what happens to users when it hits

> **Tip**
>
> Then decide whether the value justifies operating it. “We built it” is a sunk cost; “we should run it” is a decision. This checklist is short enough to run on every internal tool — including the ones an AI built for you in an afternoon.

Speaker notes